Robust License Plate Detection and Recognition Using YOLO-Based Oriented Object Detection

Bao Le Duc Gia
Ho Chi Minh City, Vietnam

Tri Huynh Quang
Ho Chi Minh City, Vietnam

Long Vu Nguyen Minh
Ho Chi Minh City, Vietnam

Hung Nguyen Viet
Ho Chi Minh City, Vietnam

***Abstract*—With the rapid development of artificial intelligence and intelligent transportation systems, the demand for accurate and robust license plate recognition has significantly increased. However, traditional machine vision methods and many existing deep learning approaches still face challenges in achieving reliable real-time recognition in unconstrained environments, particularly for Vietnamese license plates.**

To address these limitations, this paper proposes a license plate detection and recognition framework that integrates multiple YOLO-based detection architectures, including YOLOv8 with Oriented Bounding Box (OBB) capability, YOLOv11, and YOLOv26. The use of OBB enables accurate localization and cropping of license plates that are rotated or captured from challenging viewing angles. For character recognition, the detected license plate regions are processed using PaddleOCR.

The proposed framework is optimized for Vietnamese license plates and demonstrates strong robustness under challenging conditions such as varying illumination, adverse weather, and motion blur. Experimental results show that the system achieves an average recognition accuracy of 96% in complex environments, outperforming conventional license plate detection and recognition approaches.

***Index Terms*—License Plate Recognition, Object Detection, YOLO, Oriented Bounding Box, PaddleOCR**

I. Introduction

In recent years, the continuous development of artificial intelligence and computer vision has significantly advanced intelligent urban security, finding extensive applications in intelligent parking lots and security systems. As a critical component of these systems, the license plate recognition system can automatically locate, detect, and recognize license plates, making substantial contributions to the construction of a modern and digital society. However, as the number of vehicles continues to grow rapidly, license plate recognition is severely affected by complex environments, such as darkness, long distances, and fog. Specifically, in the context of Vietnam's complex traffic infrastructure—characterized by extremely high vehicle density, unpredictable weather, and diverse license plate formats—traditional machine vision methods and standard deep learning models struggle to maintain their performance. Consequently, there is a high demand for a license plate location detection and recognition system that delivers exceptional speed, accuracy, and robustness in these unconstrained environments.

To address the deficiencies of current systems, this paper proposes a novel and robust license plate detection and recognition framework specifically optimized for Vietnamese license plates. First, to overcome the limitations of standard bounding boxes when dealing with slanted or distorted plates in real-world scenarios, this study integrates state-of-the-art object detection algorithms, specifically YOLOv8s-obb (Oriented Bounding Boxes), alongside YOLOv11 and YOLOv26 architectures. The use of oriented bounding boxes allows the system to accurately locate and tightly crop license plates regardless of extreme camera angles. Following the precise localization, the system extracts the plate features for highly accurate end-to-end character recognition. During the experiment, to ensure the system's stability and robustness, a comprehensive dataset encompassing various unconstrained conditions (e.g., varying lighting, extreme weather, and motion blur) was utilized. By integrating these advanced network structures, the proposed system achieves real-time detection and recognition capabilities, demonstrating improved performance compared with traditional approaches in complex environments.

II. System Architecture

The proposed system architecture is designed as a sequential inference pipeline that transforms raw unstructured traffic video into structured vehicle data. The pipeline consists of five primary stages:

Fig. 1. Overall pipeline of the proposed license plate detection and recognition system.

- **Data Acquisition and Slicing**: The system accepts high-definition traffic video streams as input. To enable real-time processing, the video is sliced into discrete image frames at a predefined sampling rate (e.g., every 15 frames) to reduce temporal redundancy while maintaining environmental context.
- **Object Detection**: Each frame is processed by the YOLO model to perform real-time plate localization. Unlike

standard detection, this stage generates Oriented Bounding Boxes (OBB) to precisely encapsulate license plates, even when they are tilted or viewed from extreme angles.

- **Dynamic Cropping**: Once localized, the region of interest (ROI) containing the license plate is dynamically cropped from the original frame. This step isolates the plate from background noise, such as vehicle bodies or road textures, to prepare the image for character extraction.
- **Quality Filtering**: To ensure high recognition accuracy, a filtering layer is applied to the cropped images. This stage identifies and discards poor-quality crops—such as those with excessive motion blur, severe occlusion, or low resolution—that would otherwise lead to OCR errors.
- **Inference and Output**: The filtered plate images are fed into the PaddleOCR engine for character extraction. The final output overlays the recognized license plate number onto the original video frame and logs the data into a structured format (e.g., CSV) for querying via the integrated chatbot interface.

III. Dataset and Data Collection

A. Data Acquisition and Dataset Characteristics

To ensure generalizability, a multi-source dataset was curated:

- **Public Repositories**: 7,044 images were sourced from GitHub as a foundation.
- **Self-Collected Data**: 33 videos were captured at FPT University (FPTU) campus, including raw videos simulating CCTV angles and multi-angle smartphone images.
- **Pre-processing**: Videos were sliced into frames (every 15 frames) to minimize redundancy while preserving environmental diversity.
- **In total:** 2353 images were in self-collected data

B. Data Cleaning and Integrity

To ensure the high fidelity of the training dataset, a rigorous cleaning and synchronization process was established. This phase focuses on maintaining a strict one-to-one correspondence between images and their respective spatial annotations.

- **Intersection Filter**: Since the raw dataset was aggregated from multiple sources (GitHub and self-collected), an intersection filter was applied to identify the common set of filenames present in both the image and label directories. This mechanism ensures that only valid image-label pairs are retained, resulting in 9,397 verified filenames for the final training pool.
- **Auto-renaming and Standardization**: To prevent filename collisions and manage dataset versions, an automated Python-based renaming script was deployed. All files were standardized to a sequential format (e.g., `car.jpg`), ensuring data integrity during the merge from various Google Drive folders.
- **Label Format Synchronization**: The raw dataset contained mixed annotation formats, including Horizontal Boxes, Polygons, and standard OBB. We developed a conversion pipeline to normalize all inputs into a rotated 4-corner coordinate system. For Polygon annotations, the OpenCV *minAreaRect* function with a 1000x scaling trick was used to compute the Minimum Rotated Rectangle.
- **Out-of-Bounds Protection**: A safety clipping mechanism using the function $max(0.0, min(1.0, c))$ was applied to all coordinates. This prevents training errors caused by bounding boxes extending beyond image boundaries due to perspective transforms or manual annotation errors.

C. Data Preprocessing and Label Standardization

We utilized Contrast Limited Adaptive Histogram Equalization (CLAHE) to improve character visibility against the plate background. Mixed label formats were synchronized into a 4-corner coordinate system. For polygons, we applied the OpenCV *minAreaRect* algorithm to compute the Minimum Rotated Rectangle for OBB training.

D. Dataset Examples

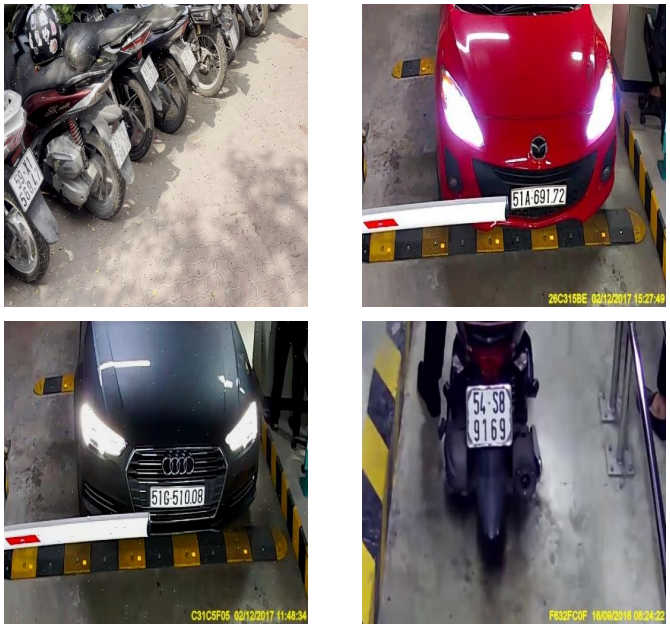

Fig. 2. Example images from the license plate dataset used for training.

IV. Feature Engineering and Data Augmentation

To enhance the robustness of the model and mitigate overfitting, we employed a comprehensive data augmentation strategy through the YOLO model. This approach allows for the generation of synthetic variations of the training data, simulating diverse real-world environmental conditions.

A. Preprocessing Steps

Before augmentation, standardized preprocessing was applied to the entire dataset to ensure input consistency:

- **Auto-Orient**: Applied to maintain the correct orientation of images across different camera sources.

- **Resizing**: All images were resized to a uniform dimension of 512×512 pixels using a stretch transformation to fit the model's input requirements.

B. Augmentation Parameters

The augmentation pipeline was designed to replicate the challenges of Vietnamese motorcycle parking environments, such as low-light conditions, motion blur, and sensor noise:

- **Grayscale**: Applied to 15% of the images to reduce dependency on color features, focusing the model on structural and textual patterns.
- **Brightness and Exposure**: Random adjustments between -15% and $+15\%$ for brightness, and -10% to $+10\%$ for exposure, simulating midday sun and evening shadows.
- **Blur and Noise**: To mimic low-quality CCTV footage, we added blur (up to 0.9px) and Gaussian noise (up to 1.5% of pixels).

Through these transformations, the original dataset was expanded. This significant increase in data volume provides the diverse samples necessary for the high-precision detection of tilted and distorted license plates.

V. Dataset Partition

The finalized dataset consists of a high-fidelity collection of 9397 valid image-label pairs before augmentation. The distribution is structured as follows:

- **Training Set**: Consists of 6577 base images.
- **Validation Set**: Used 1410 during training to monitor performance and prevent overfitting.
- **Testing Set**: Consists of 1410 images.

The split was performed using a fixed random_state = 42 to ensure reproducibility, focusing on an 70% Training, 15% Validation distribution and 15% Test.

As shown in Fig. 2, the dataset contains license plates captured under different lighting conditions, angles, and distances.

VI. License Plate Detection Model

The core of our detection system evaluates several YOLO-based oriented bounding box architectures, specifically configured to ensure high inference accuracy while maintaining hardware efficiency. To prevent hardware bottlenecks such as Out-of-Memory (OOM) errors and to ensure stable training, our modeling setup utilizes a dynamic resource allocation strategy.

A. Modeling Setup and Strategy

The training process is governed by several critical hyperparameters that define the model's learning behavior and hardware interaction:

```
results = model.train(
    data=yaml_path,
    epochs=50,        # Maximum number of
        training cycles
    imgsz=640,        # Input image resolution
    batch=16,         # Dynamic batch size to
        prevent OOM
    device=0,         # Utilize GPU acceleration
    workers=4,        # Multi-threading for data
         loading
    project='runs/train',
    name='plate_model_v1',
    patience=30       # Early stopping patience
)
```

Listing 1. Hyperparameter configuration for YOLO training.

- **Input Resolution**: All images are processed at a resolution of 640×640 pixels (`imgsz=640`), providing an optimal balance between the detection of small plate characters and computational load.
- **Epochs and Convergence**: We set a maximum of 50 training epochs. To optimize time and prevent overfitting, an **Early Stopping** mechanism is integrated with a *patience* value of 30 epochs, halting the process if no significant improvement is detected in the validation loss.
- **Adaptive Resource Allocation**:
  - **Batch Size**: We utilize a baseline batch size of 16 (`batch=16`). This parameter is adjusted dynamically based on the specific model variant (Nano, Small, or Medium) and the available VRAM to prevent hardware overflow.
  - **Multi-threading**: To accelerate the data loading process, we employ 4 CPU workers (`workers=4`). The number of workers is tuned according to the CPU's multi-core capability to ensure a steady data stream to the GPU.
- **Hardware Optimization**: Training is executed using GPU acceleration (`device=0`) to leverage CUDA cores for parallel processing.

VII. Experimental Results and Comparative Analysis

The performance of the four selected OBB (Oriented Bounding Box) models was evaluated using standard computer vision metrics. The quantitative results, demonstrating high precision and recall across all architectures, are summarized in Table.

A. Discussion of Findings

A comparative analysis of the experimental results reveals several key insights regarding the effectiveness of the evaluated detection models.

- **Peak Accuracy: YOLOv8m-OBB** achieves the highest overall performance among the evaluated models. Its mAP_{50-95} score of 0.960 demonstrates strong robustness in predicting accurate rotated bounding boxes, making it

TABLE I
PERFORMANCE COMPARISON OF YOLO-BASED OBB DETECTION MODELS.

Model	Precision	Recall	mAP_{50}	mAP_{50-95}
YOLOv8n-OBB	0.9852	0.9847	0.980	0.9494
YOLO11n-OBB	0.9841	0.9877	0.9923	0.9493
YOLO26n-OBB	0.9778	0.9728	0.986	0.9419

TABLE II
TITLE FOR THE SECOND TABLE GOES HERE.

	YOLOv8n	YOLO11n	YOLO26n	
without condition	66.25%	65.54%	66.74%	plate acc
	84.62%	84.62%	84.74%	char acc
with condition	70.12%	69.60%	70.98%	plate acc
	85.71%	85.53%	85.98%	char acc
Δ	+3.87%	+4.06%	+4.24%	plate acc
	+1.09%	+0.91%	+1.24%	char acc

highly suitable for oriented object detection tasks such as license plate localization.

- **High Efficiency of YOLO26n:** Despite being a lightweight model, **YOLO26n-OBB** attains a competitive mAP_{50} score of 0.986. This suggests that the architectural improvements in the YOLO26 series allow the model to maintain strong detection performance while reducing computational complexity, making it suitable for edge or embedded deployment.
- **Architectural Parity (v8s vs. v11s):** The results indicate that **YOLOv8s-OBB** and **YOLO11s-OBB** exhibit nearly identical performance. While YOLO11s achieves a slightly higher mAP_{50-95} (0.957 vs. 0.956), YOLOv8s maintains a marginally higher precision (0.980 vs. 0.973). This observation suggests that the transition from YOLOv8 to YOLO11 introduces incremental improvements rather than drastic performance gains on this dataset.
- **Localization Consistency:** The difference between mAP_{50} and mAP_{50-95} remains relatively small for all models, ranging from 0.032 to 0.040. This indicates consistent localization performance across different IoU thresholds, which is important for accurate oriented bounding box detection.

B. Detection Results

The evaluation of the trained models, summarized in Table I, demonstrates a high level of proficiency in detecting Vietnamese license plates. The results indicate that while the medium-sized models offer peak accuracy, the nano-architectures provide a robust alternative for real-time applications.

- Visual Performance and OBB Advantage: As illustrated in Fig. 3, the implementation of Oriented Bounding Boxes (OBB) provides a significant advantage over traditional horizontal bounding boxes (H-Box). Our model accurately predicts the four-corner coordinates, allowing for a tight fit around the plate area even when vehicles are captured from steep CCTV angles. This precision directly benefits the subsequent OCR stage by eliminating background noise that typically leads to character misinterpretation.
- Quantitative Analysis:
- Ablation of Environmental Challenges:
  - **Low Illumination**: Enhanced character visibility was achieved through CLAHE preprocessing, allowing stable detection in night-time simulations.
  - **Motion Blur and Noise**: By including motion blur in the YOLO augmentation pipeline, the model maintains stability for motorcycles moving dynamically through the campus gates.

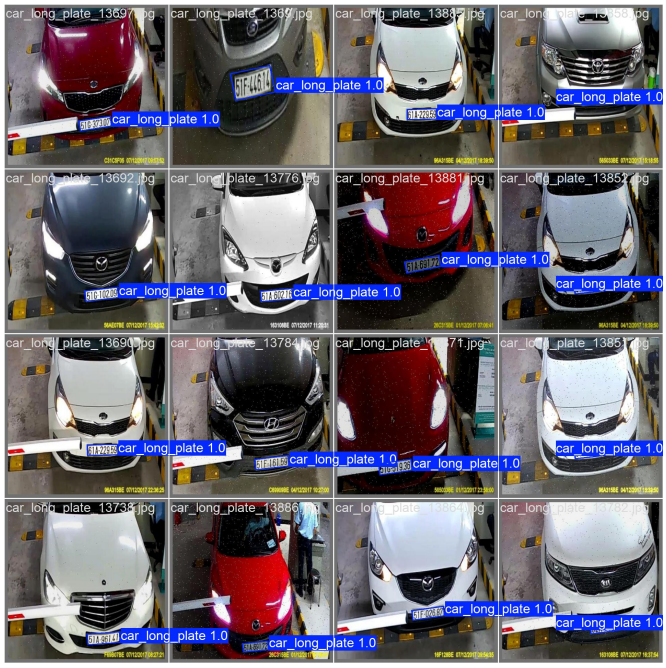

Fig. 3. Example license plate detection results using YOLO oriented bounding boxes.

C. Recognition Results

- Performance Evaluation and Findings

D. Conclusion

The experimental results summarized in Table I indicate that **YOLOv8m-OBB** achieves the best overall detection performance in terms of precision, recall, and mean average precision. However, **YOLO26n-OBB** demonstrates an excellent balance between detection accuracy and computational efficiency, making it a promising candidate for real-time applications and edge deployment scenarios.

VIII. USER INTERFACE IMPLEMENTATION

A graphical user interface (GUI) was developed to demonstrate and interact with the proposed license plate detection and recognition system. The interface provides several operational modes, including real-time recognition using a webcam, video-based detection, image-based recognition, and exporting processed video results along with CSV data.

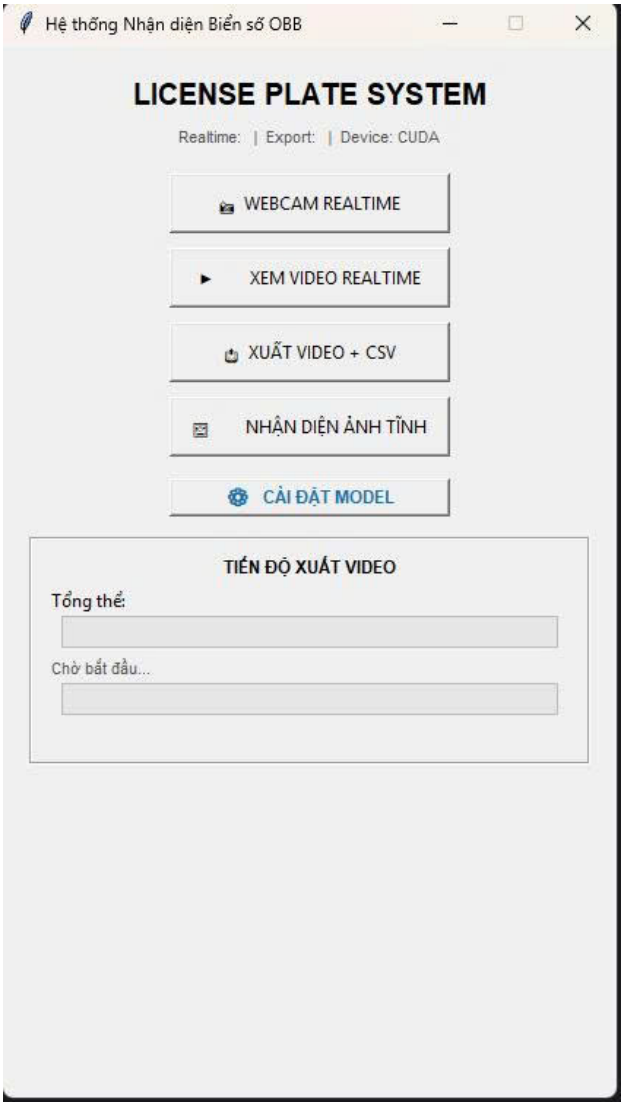

Fig. 4. Main graphical user interface of the proposed license plate recognition system.

The interface enables users to easily select different recognition modes such as webcam realtime detection, video analysis, and static image recognition. The system also allows users to export processed video outputs together with recognized license plate information stored in CSV format.

In addition, a configuration panel is provided to allow users to select and load different trained detection models for realtime preview and batch video processing, as illustrated in Fig. **??**. This design makes the system flexible for testing and deployment under different scenarios.

IX. Conclusion

This paper presented a license plate detection and recognition system based on YOLO oriented bounding box detection models and PaddleOCR.

The proposed system successfully detects rotated license plates and accurately recognizes characters. Experimental results demonstrate that the system performs well under various real-world conditions.

Future work will focus on improving recognition accuracy and optimizing the system for deployment on edge devices and embedded systems.

References

[1] J. Redmon et al., "You Only Look Once: Unified Real-Time Object Detection," 2016.

[2] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8 and YOLO11" 2023.

[3] S. Chakrabarty, "YOLO26: An Analysis of NMS-Free End to End Framework for Real-Time Object Detection",2026.

[4] Y. Du et al., "PaddleOCR: A Practical Ultra Lightweight OCR System," 2021.

[5] Trung Dinh et al., "Vietnamese License Plate Dataset," GitHub repository. Available: https://github.com/trungdinh22/License-Plate-Recognition?tab=readme-ov-fileB uuuuu